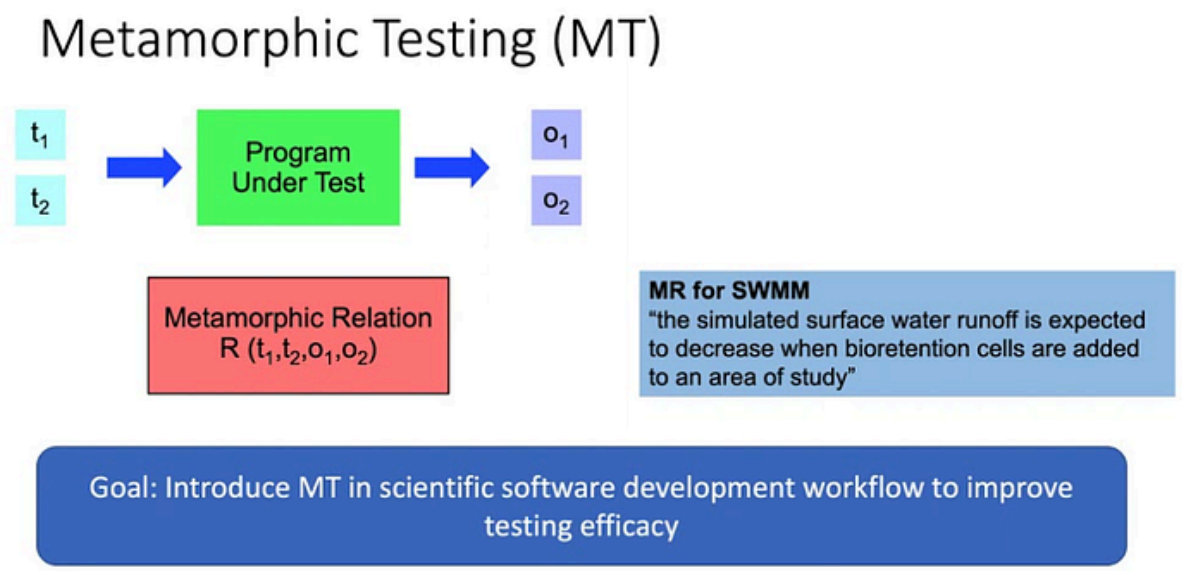


In the ever-evolving landscape of software development, ensuring the reliability and correctness of software is paramount. Traditional testing methods, while effective, often stumble when faced with complex, non-deterministic, and undecidable problems. This is where Metamorphic Testing (MT) enters the scene, offering a novel approach to overcome these challenges.

Get Devender Sharma's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

Remember me for faster sign in

Let’s have a look at some concepts of Metamorphic Testing, its methodology, applications, and examples, providing a comprehensive understanding of this innovative testing paradigm.

Understanding Metamorphic Testing i.e. MT

Metamorphic Testing is a software testing technique that addresses the issue of test oracle problems. The test oracle problem arises when it is difficult to determine the expected outcome of a test case. MT circumvents this by focusing on the relations between the outputs of multiple executions of the program under test, using altered inputs. These relations are known as Metamorphic Relations (MRs).

The Core Concept

The core idea of MT is not to test individual outputs for correctness but to check if the output relations remain invariant under changes in input. This approach is particularly useful in situations where the correct output is unknown or difficult to ascertain.

Metamorphic Relations (MRs)

MRs are the heart of Metamorphic Testing. They are properties or relations that should be preserved between the outputs of multiple executions of the program with different inputs. Identifying appropriate MRs is crucial for the effectiveness of MT.

Methodology of Metamorphic Testing

1. ***Identification of MRs:*** The first step is to identify suitable MRs for the software under test. These relations are based on the understanding of the software's functionality and domain properties.
2. ***Selection of Test Cases:*** Initial test cases are generated without the concern of knowing the exact output.
3. ***Generation of Follow-Up Test Cases:*** Based on the MRs, new test cases (follow-up test cases) are generated by altering the initial test cases.
4. ***Execution of Test Cases:*** Both the initial and follow-up test cases are executed.

5. ***Verification of MRs:*** The final step is to check if the MRs hold for the outputs of the initial and follow-up test cases. If an MR is violated, a defect is likely present.

Applications of Metamorphic Testing

Metamorphic Testing has found applications in various domains, particularly where traditional testing methods fall short:

- ***Machine Learning and AI:*** In AI algorithms, especially in machine learning models, predicting the exact output is often impractical. MT can be used to verify the consistency and correctness of these models.
- ***Web Services and APIs:*** MT can test web services and APIs, especially when the services are black-box and the internal workings are unknown.
- ***Scientific Computing:*** In scientific applications where the results are complex and exact values are not known, MT helps in validating the software's functionality.

Examples of Metamorphic Testing

Example 1: Testing a Sorting Algorithm

- ***MR Identification:*** A basic MR for a sorting algorithm could be that if you shuffle the elements of the input list, the output (sorted list) should remain the same.
- ***Test Case Generation:*** Generate a random list of numbers (initial test case) and a shuffled version of this list (follow-up test case).
- ***Verification:*** After sorting both lists, the outputs should be identical. If not, there is a flaw in the sorting algorithm.

Example 2: Machine Learning Model

- ***MR Identification:*** For a regression model, an MR could be that multiplying all input features by a positive constant should result in the output being multiplied by the same constant.
- ***Test Case Generation:*** Create an input dataset and a modified version where all features are scaled by a constant.

- ***Verification:*** Compare the outputs; they should be scaled consistently according to the MR. Deviations might indicate issues in the model or data processing.

Metamorphic Testing represents a significant shift in software testing paradigms, especially for scenarios where traditional testing approaches are inadequate. By focusing on the relationships between different outputs, MT provides a robust mechanism for validating software in complex, uncertain, and non-deterministic domains.

Metamorphic Testing in a Nutshell

23 Dec, 2021
Tags: software-testing

"Building the product right versus building the right product" is a popular cliché used to describe the product-market fit. Software

testing focuses on "building the right product". There are analogies in the mathematical world, where every claim is validated by proof. The level of rigour required in mathematics is that the proof must handle every case to be acceptable. In the world of software testing, nothing stops us from considering every case, no matter how trivial. However, the difficulty lies in identifying every possibility, which can be exponential in some cases.

Traditional testing requires crafting a few examples (input/output pair) and checking that the output matches our expectations (ground truth) [1]. There are situations where obtaining test examples is difficult. Traditional testing suffers from the Oracle problem, or insufficient coverage in some cases. Hence, impacting the applicability of traditional testing in those areas. Handling these issues would require metamorphic testing.

Verification versus Validation

There is usually an upfront cost for writing tests for the application. Due to these costs, some managers have been hesitant to invest in testing. There is a preponderance of evidence about the value that testing provides to an organization. However, the level of testing can vary across organizations. Testing is considered an afterthought by some schools of thought, while test-driven development is considered the only way to go by others. Rather than focus on both extremes, my take is that the programmer should write enough tests to give confidence that features can be added without breaking existing functionalities.

When done rightly, the benefits of a testing culture include the following:

A reduction in the time spent troubleshooting bugs.
A quality mentality that spreads among team members in the organization.

Anecdotal evidence suggests that Software engineers who create new features while writing tests tend to have lower technical debts in their source code.

Software testing can take the following form:

> **Verification** is a series of checks on the source code, design documents, and architecture to ensure the quality of the software via reviews and inspection. This entails checking if the software matches its planned requirements.
> **Validation** is checking if the software meets the needs of the customer.

In a game-theoretic sense, we can have a set of testers and a set of programmers which may overlap and act in good faith. The programmers and testers are actors in our fictitious game. Let us restate our goals in the following phrases.

> The goal of a programmer is to write code that can pass the test. Testers write tests that can fail and reveal problems.

One may be tempted to ask the following questions.

> Is there any value in a passing test?
> If so, what information derives from a passing test [2]? One may think that passing tests is useless. However, a passing test assures us that the code path is working as intended, but it tells us nothing about its behaviour in any untested code path. Hence, stressing the need for higher test coverage. Metamorphic testing comes to the rescue.

What is Metamorphic Testing?

Metamorphic testing (MT) reduces the need for an Oracle by using the relationship between two or more inputs (derived from the original input) and their expected outputs. These relationships are more general and can help increase test coverage. They are also known as metamorphic relations (MR). However, if any metamorphic relation is violated, then a bug has been detected. MT helps in test case generation and result verification [2]. MT can provide a framework for robust testing of edge cases and failure modes.

Metamorphic testing is a widely used methodology in various forms. However, only recently did academics formalize it. Metamorphic testing can help test a program when the expected output (oracle) is unavailable. However, we require a proxy for this measure if the ground truth is absent. There are numerous applications of metamorphic testing in machine learning [4], compiler development [7], and bioinformatics [6].

Metamorphic testing verifies the output of a test case based on slight perturbation of the input while retaining the same information. These are typical problems that are inherently lacking ground truth for traditional testing. These include:

> Testing the effectiveness of a text-to-speech (TTS) processor.
> Is the result from a search query correct?
> Hence, we will provide examples of metamorphic relations in this blog.

A possible metamorphic relation in graph-theoretic problems is the use of symmetry. Assume that we have an undirected graph in which every node has a bidirectional link with the same weight. We can be sure that

$$Dist(G,a,z)=Dist(G,z,a) \quad (1)$$

Where

$Dist(G,x,y)$

is the distance between node,

x

and node,

y

in a graph,

G

. Similarly, finding the minimum of two values should be independent of the order

$$min(a,b)=min(b,a)$$

(2)

More examples of metamorphic testing

- Comparing diffs of binary
- Verifying the shortest path algorithm
  - In a single-source shortest path, it can be checked that every node is relaxed, given that we have the cost of the node.
- Checking the order of execution, e.g. happen-before relations in Distributed System

Automated log analysis

Here is an example of traditional testing.

```
def example1():
    p=5
    assert(p==5)
```

Notice that we have only examined a point in the problem space, which is in contrast to generative testing, which is the basis for property-based testing. Metamorphic testing is related to Property-based testing, as it is a generative testing paradigm.

```
def example2()
    p=random_point()
    assert(p>5)
```

The benefit of generative testing is higher coverage. However, we trade specificity for generality [1].

The challenges of using generative testing:

Finding appropriate relationships.
Specific versus General tradeoff.
Specific has more information, while general can work across multiple use cases, but has minimal information

Possible Use cases

In a Neural network, if you have done some convolutions, you may check if the translational matrices obey the properties of a typical translation matrix.
In tabulation, if you are dealing with human ages, it is possible to check if ages are between 0 and 115 as humans have a limited lifespan which is a universal feature of humans. In some applications, you may want to verify whether the user has reached the acceptable age to use the application.
If you know the call graph of processes on a browser, it is possible to predict the loading order of the page and use it to identify bugs.
Checking time bounds violations in distributed systems.
Investigating logs of an application to detect bugs.
Creating a parser for CSS style files to identify usability bugs by checking printer-safe, colour-blind (safe colour) codes in the CSS files. This can identify accessibility bugs.
Checking UI placement (snapshot testing).
Validating a clustering algorithm without ground truth. Deciding on the optimal number of clusters can be challenging and ill-posed. Formulate adequacy criteria based on the structure of the data. A solution may look unintuitive, but it is an idea for a specific use case. See more information in the paper [4].

Advantages of MT

Here are some advantages of MT[2]:

Simple
Easy to implement
Easy to automate
Low cost

Case studies in information retrieval

Here are examples of metamorphic relations in evaluating the usefulness of a search engine [3, 5].

Equivalence: A repeated search query should return the same element, independent of ordering. For example, a query "houses in Vancouver" should yield results similar to "Vancouver houses".
Equality: If sorted by relevance, a search query will return the same result for the same query. For example, if I search in the search engine twice within a short time. The results should be identical.
Subset: A subset of a query should have fewer results than the original query. For example, a query "houses in British Columbia" should return more results than a query "houses in Vancouver". This is because Vancouver is a city in the province of British Columbia.

Disjoint: Irrelevant responses should not be returned. In the query "houses in Vancouver", we should not see any houses from Saskatchewan in the search result.

Complete: A universe is the union of all of its components. For example, a query "houses in British Columbia" will display a combination of houses in every city in the province of British Columbia.
Difference: A query "houses in British Columbia, not Vancouver" should not return any results for houses listed in Vancouver.

Misconceptions around MT Relations

There are things that can impact how MT is used in practice. Even though violating the bound results in errors, not all necessary properties are MR, e.g. -1 *= sin(x) *= 1 even though sine is not an MT.

"MR targets multiple inputs and their expected output."
"Not all MRs separate into input-only and output-only sub-relations."

"Not all MRs are equality relations."
"MR can work with or without Oracle."

Applying metamorphic testing in practice

Figure out how inputs are modified to effect a predictable pattern in the output.
Due to sufficiency versus necessary constraints, do not assert but rather warn.
Metamorphic testing should not replace standard testing (unit testing, interaction testing). MT is meant to augment existing test suites by increasing test coverage.

Conclusions

Metamorphic testing should serve as a secondary testing paradigm in your tool set. It is reasonable to see metamorphic testing as sanity checks based on the property of the (input/output) problems. One rule of thumb for applying metamorphic testing is to study successful case studies in the industry and adapt to your specific needs. Finally, creating Metamorphic relations is a combination of both art and science.

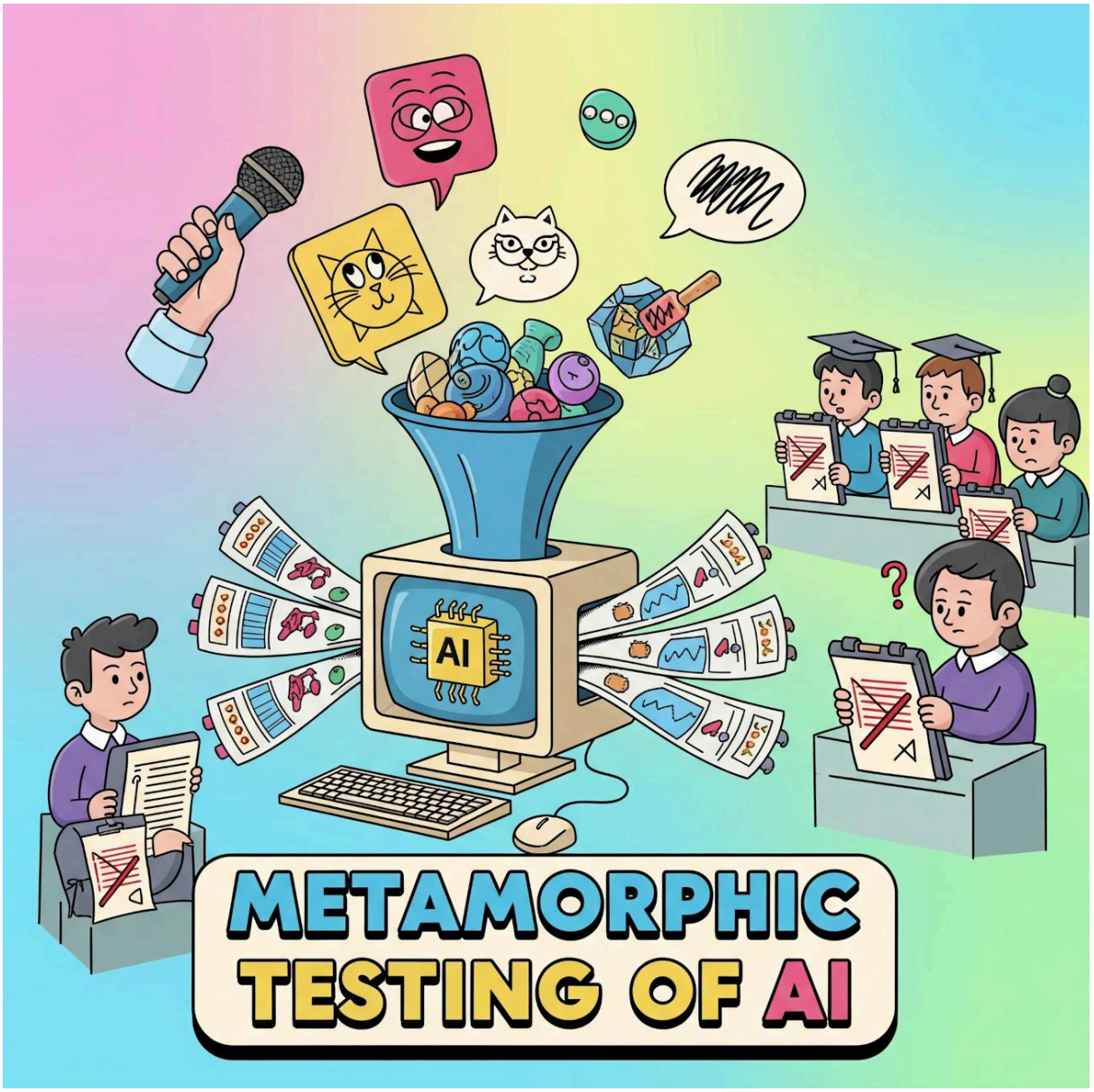
AI is not as easy to test as traditional software. Think of attempting to predict each and every possible result for an AI that's constantly learning and evolving, like an advanced image recognition system. It is incredibly hard, if not impossible, to formally specify what the "correct" answer looks like for every input. This, of course, is the "**oracle problem**" in AI testing – we frequently do not have a clear, unambiguous "oracle" to tell us whether the AI output is 100% correct.

This is where a smart technique named Metamorphic Testing (MT) comes in. Where traditional approaches may seek a single correct answer, MT examines how an AI's outputs change when its input changes in some predictable way.

Key Takeaways

- Metamorphic Testing solves the AI “oracle problem” by focusing on the relationship between input changes and output behavior. Instead of needing exact answers, it validates if outputs shift logically.
- Start with a base test input and output, define a rule for how outputs should change, apply a transformation, and check if the expected relationship holds. This verifies AI logic without needing a ground truth.
- **Common types:**
    - Invariance: Output stays the same despite minor input changes.
    - Increasing: Input changes logically raise output scores or probabilities.
    - Decreasing: Input changes logically lower output scores or probabilities.
- **Challenge**: Defining meaningful metamorphic relations requires domain expertise and abstract thinking.

Let’s delve deeper into Metamorphic Testing.



What is Metamorphic Testing?

If traditional testing struggles to give us a definitive “correct” answer for every AI output, how exactly does Metamorphic Testing (MT) get around this challenge?

At its heart, Metamorphic Testing isn't about finding a single right answer. Instead, it's about checking the **relationships** between inputs and outputs. Think of it like this: if you have a recipe for a cake, and you bake it according to the instructions, you get a cake. Now, if you *double* all the ingredients, you should get *two* cakes (or one much larger cake). You might not know the exact weight or perfect texture of that bigger cake without a reference, but you know it should relate predictably to the first one. MT applies this same logic to AI.

Here's how it generally works:

- **Start with a 'Source' Test:** You begin with a regular test case – an input you feed into your AI model, and the output it produces. For instance, if you're testing an image recognition AI, you might show it a picture of a dog, and it correctly identifies "dog."
- **Define a 'Metamorphic Relation':** This is the clever part. A "metamorphic relation" is a rule that describes how a change in the input should predictably affect the output. It's a fundamental property that *must* hold true for your AI. For our dog image example, a metamorphic relation could be: "If I slightly rotate the image of the dog, the AI should still identify it as a dog, and its location in the image should shift accordingly." You're not saying "it must be identified as *exactly* a dog," but rather, "if I do X to the input, Y *must* happen to the output's meaning or structure."
- **Create a 'Follow-up' Test:** Based on your metamorphic relation, you transform your original input to create a new, "follow-up" test case. So, for our dog example, you'd create a new image where the original dog picture is rotated by 15 degrees.
- **Verify the Relationship:** Finally, you feed this transformed input into your AI and check if the relationship described by your metamorphic relation holds true. Does the AI still say "dog" for the rotated image? Do the detected coordinates of the dog also appear to be rotated in the output? If the expected relationship doesn't hold, then you've found a potential bug or inconsistency in your AI, even without knowing the "perfect" answer for that specific rotated image.

Why is Metamorphic Testing Needed?

Let us review the reasons why we need metamorphic testing in AI.

Solving the “Oracle Problem”

Imagine trying to build an AI that can perfectly describe any photograph you show it. How do you test if its description is “correct”? There’s no single, universally agreed-upon answer for every image. This challenge, known as the “oracle problem,” is a major roadblock for traditional testing. With complex AI models – like those used for facial recognition, understanding human language, or even predicting stock market trends – it’s often impossible to have a pre-defined, perfect answer for every conceivable input. MT sidesteps this entirely. Instead of needing a perfect answer, it asks: “If I slightly alter this photo, does the AI’s description change in a way that makes sense?” This shift in perspective is revolutionary for AI testing.

Handling AI’s Unique Nature

Unlike traditional software that often follows strict, predictable rules, AI models can be a bit more fluid and, well, *intelligent*. They might produce

slightly different outputs for very similar inputs, or their decision-making process can be incredibly complex. This “fuzziness” and the sheer number of possible inputs make it extremely difficult to cover all bases with standard testing. MT helps here because it doesn’t try to test every single possibility. It focuses on general principles of how the AI *should* behave when inputs are related, even if the exact output isn’t perfectly predictable.

Building Stronger, Smarter AI

Beyond just finding bugs, MT plays a vital role in making AI systems more reliable and trustworthy. It helps us uncover:

- **Vulnerabilities:** Are there tiny changes to an input that can trick the AI into making a completely wrong decision? For example, adding a few “invisible” pixels to a stop sign image might make a self-driving car AI think it’s a yield sign. MT can expose these weaknesses. Read: Top 10 OWASP for LLMs: How to Test?
- **Biases:** If an AI is trained on biased data, it might perform poorly or unfairly for certain groups. MT can help reveal these biases by applying predictable transformations to inputs (e.g., changing the skin tone in an image) and observing if the AI’s behavior changes unexpectedly.
- **Edge Cases:** These are unusual or extreme situations that the AI might not have encountered during its initial training. MT’s ability to generate related test cases helps uncover how the AI behaves in these tricky scenarios, leading to models that perform better in the real world.

Most Used Metamorphic Relations to Test AI

The three most widely used categories of metamorphic relations are:

Invariance (Things Should Stay the Same)

This is perhaps the most common and intuitive type. An invariance relation means that if you make a specific change to the input, the core output or its meaning *should not change at all*. The AI should be "invariant" or unaffected by that particular transformation.

- **Example 1: Language AI and Synonyms:** If an AI classifies text (like deciding if a customer review is positive or negative), changing a word to its synonym should ideally not change the overall sentiment. If "This service was *excellent*!" is positive, then "This service was *great*!" should also be positive. The AI's classification should remain *invariant*.
- **Example 2: Image Recognition and Minor Adjustments:** If an AI identifies a "car" in a picture, and you slightly brighten the image, or add a tiny, unnoticeable bit of digital "noise," the AI should still identify it as a "car". The identification should be *invariant* to these minor, irrelevant changes.
- **Example 3: Search Engines:** If you search for "best Italian restaurants near me" and then search for "best Italian eateries near me", the core results should be very similar, if not identical, assuming "restaurants" and "eateries" are treated as synonyms. The search results should be *invariant* to the synonym substitution.

Increasing (Outputs Should Go Up)

This type of relation checks if a change in the input logically leads to an increase in a certain output value or probability.

- **Example: Credit Risk Assessment:** Imagine an AI that predicts the likelihood of someone defaulting on a loan. A metamorphic relation could be: “If a person’s income significantly *decreases*, their predicted risk of defaulting on a loan should *increase*.” You’re expecting a direct, upward relationship between one factor and another.
- **Example: Spam Detection:** If an email is initially classified as “not spam,” but you then add many known “spammy” keywords to its content, the probability of it being classified as spam should *increase*.

Decreasing (Outputs Should Go Down)

Opposite to “increasing”, this relation checks if a specific input change should logically lead to a decrease in an output value or probability.

- **Example: Credit Risk Assessment (continued):** Following the previous example, if a person’s credit score significantly *improves* (meaning they’re more financially stable), their predicted risk of defaulting on a loan should *decrease*.
- **Example: Content Moderation:** If an AI flags inappropriate content, and you remove several offensive words or phrases from a text, the likelihood of that text being flagged as inappropriate should *decrease*.

Challenges in Making Metamorphic Testing Work for AI

There are some practical hurdles and important considerations you need to keep in mind when trying to put it into practice.

Crafting the Right “Rules” (Metamorphic Relations)

This is arguably the trickiest part. Coming up with those clear, logical “metamorphic relations”, the rules that say how your AI’s output should predictably change when the input changes, requires a deep understanding. It’s not just about knowing how the AI works internally (which can be a black box sometimes!), but also about knowing how it *should* behave in the real world.

- **You Need Experts:** This often means involving people who truly understand the subject matter the AI is dealing with. For a medical AI, you’d need doctors; for a financial AI, financial experts. They’re the ones who can tell you, “If this patient’s blood pressure goes up just a little, their diagnosis shouldn’t suddenly flip from healthy to critically ill”.

- **It’s About Logic, not Code:** You’re defining a logical rule about behavior, not writing a piece of code that defines an exact output. This abstract thinking can be a different mindset for many testers.

Automating the Process

Once you have your rules, you don’t want to manually create thousands of altered inputs and then manually check if the AI’s output follows the rule. That would defeat the purpose!

- **Generating New Tests:** You need tools or clever programming to automatically take an original input and transform it according to your metamorphic relation. For example, if your rule says “rotate the image,” you need a way for your system to automatically create those rotated versions.
- **Checking the Results:** Similarly, you need an automated way to compare the original output with the transformed output and see if the metamorphic relation holds true. This can be complex, especially if the relationship is subtle or involves changes in meaning rather than just numbers.

Avoiding False Alarms (False Positives/Negatives)

When you set up these rules, you might run into situations where the test indicates a problem, but there isn’t one (a “false positive”), or worse, where a real problem exists, but the test doesn’t catch it (a “false negative”).

- **Too Strict Rules:** If your metamorphic relation is too rigid, even minor, acceptable variations in AI behavior might trigger an “error”. This leads to wasted time chasing non-existent bugs.
- **Too Loose Rules:** If your rules are too vague, they might miss genuine inconsistencies or problematic behaviors. It’s a delicate balance to strike. Carefully crafting and refining your metamorphic relations is key to making sure the tests are useful and reliable.

Fitting into Your Current Workflow

Developing AI often involves rapid iteration and specific development practices (like CI/CD). Integrating a new testing approach like MT seamlessly into these existing routines can be a challenge.

- **Setup Time:** Getting the initial setup for MT (defining rules, building automation) can take time and resources.
- **Running the Tests:** How often do you run these tests? Do they run automatically every time a developer makes a change? Ensuring MT becomes a natural part of your AI development cycle, rather than a separate, burdensome task, is crucial for its long-term success.

Using AI-based Tools for Metamorphic Testing

When it comes to test automation, you can find tools in the market that can help you with it. However, most of them aren’t intelligent enough to help you

check MT relationships. While none of the tools available is inherently capable of validating MT right off the shelf, you can definitely take their assistance.

testRigor is an interesting candidate for Metamorphic Testing because of its core philosophy: *plain English test steps and AI-driven element identification, aiming for ultra-stable tests.* Its capabilities allow you to implement many MT strategies.

The key is to use testRigor's ability to:

- **Interact with your AI's interface (UI or API):** testRigor is excellent at interacting with web, mobile, mainframes, and even desktop UIs, and it also supports API testing. This is crucial for sending your original and transformed inputs to the AI.
- **Use "Reusable Rules" (functions):** This is one of the way's for you to define your metamorphic relations. You can create reusable rules that encapsulate the transformation logic and the verification logic. Read: How to create AI-based reusable rules using testRigor?
- **Data-Driven Testing:** You can use datasets to manage your original inputs and potentially parameters for your transformations.
- **Plain English assertions and Vision AI:** testRigor's natural language processing (NLP) and visual AI capabilities allow you to make powerful assertions about the AI's output, even if it's visual or complex text. These assertions can be used as the rules that validate if your output is the same despite the changes in the input.

Here's a great examples-based guide demonstrating how testRigor simplifies the testing of AI features – AI Features Testing: A Comprehensive Guide to Automation.

Conclusion

If you’re involved in developing, deploying, or even just working with AI, it’s time to seriously consider integrating Metamorphic Testing into your toolkit. It provides a brilliant solution to a tough problem: how do you test an AI when you don’t always know what the “perfect” answer should be for every possible input? By focusing on logical relationships – how changes in input should predictably affect output – MT cleverly bypasses this “oracle problem.” This unique approach allows us to delve deeper into an AI’s behavior, helping us build models that are not just accurate sometimes, but truly robust, consistent, and dependable.

Metamorphic testing

Article
Talk
Read
Edit
View history
Tools
Appearance hide
Text
Small
Standard
Large
Width
Standard
Wide
Color
Automatic
Light
Dark
From Wikipedia, the free encyclopedia

Metamorphic testing (MT) is a property-based software testing technique, which can be an effective approach for addressing the test oracle problem and test case generation problem. The test oracle problem is the difficulty of determining the expected outcomes of selected test cases or to determine whether the actual outputs agree with the expected outcomes.

Metamorphic relations (MRs) are necessary properties of the intended functionality of the software, and must involve *multiple* executions of the software. Consider, for example, a program that implements sin x correct to 100 significant figures; a metamorphic relation for sine functions is "sin $(\pi - x) = \sin x$". Thus, even though the expected value of sin x_1 for the source test case $x_1 = 1.234$ correct to the required accuracy is not known, a follow-up test case $x_2 = \pi - 1.234$ can be constructed. We can verify whether the actual outputs produced by the program under test from the source test case and the follow-up test case are consistent with the MR in question. Any inconsistency (after taking rounding errors into consideration) indicates a failure[1]:31 of the program, caused by a fault[1]:31 in the implementation.

MRs are not limited to programs with numerical inputs or equality relations. As an example, when testing a booking website, a web search for accommodation in Sydney, Australia, returns 1,671 results; are the results of this search correct and complete? This is a test oracle problem. Based on a metamorphic relation, we may filter the price range or star rating and apply the search again; it should return a subset of the previous results. A violation of this expectation would similarly reveal a failure of the system.

Metamorphic testing was invented by T.Y. Chen in a technical report[2] in 1998. Since then, more than 750 papers have been published by international researchers and practitioners, working further on the theory and practice of MT in real-life applications.[3] Some examples include web services,[4] computer graphics,[5] embedded systems,[6] simulation and modeling,[7] machine learning,[8] decision support,[9] bioinformatics,[10] components,[11] numerical analysis,[12] compilers,[13] and even quantum computing[14] The first major survey of MT was conducted in 2016.[15] It was followed by another major survey in 2018,[16] which highlights the challenges and opportunities and clarifies common misunderstandings.

Although MT was initially proposed as a software verification technique, it was later developed into a paradigm that covers verification, validation, and other types of software quality assessment.[17] MT can be applied independently, and can also be combined with other static and dynamic software analysis techniques such as proving and debugging.[18]

In August 2018, Google acquired GraphicsFuzz, a startup from Imperial College London, to apply metamorphic testing to graphics device drivers for Android smartphones.[19][20]